

Taller conceptos Generales Compiladores

① ¿Qué es un compilador y cuál es su función principal?

R/ En palabras básicas es un programa que toma el código que se escribe en algún lenguaje de programación como pueden ser C, Java, Python, etc, y lo interpreta para poder transformarlo en lenguaje máquina, es decir, código binario (1, 0) y así la máquina puede ejecutar en un archivo que pueda entender, todo el programa.

El compilador es el que hace de puente entre lo que uno escribe y lo que puede procesar el hardware.

Además de traducir, también revisa que el código este bien escrito, es decir, que no tenga errores de escritura, estructura y lógica.

$$\begin{matrix} C \\ Java \\ Python \end{matrix} \xrightarrow{\text{compilador}} \begin{matrix} \text{Código} \\ \text{Máquina} \end{matrix} (0,1)$$

② ¿Cuál es la diferencia entre un compilador y un intérprete?

R/ El compilador toma el programa completo, lo analiza y en base a ello genera un archivo aparte. Cuando este archivo ya esta creado ya no hay necesidad de volverlo a compilar ni necesitar el código fuente haciendo posible que este pueda ser conido cualquier cantidad de veces.

El intérprete no genera nada aparte, lee el código línea por línea y ejecuta paulatinamente. Lo compilado es mucho más rápido, pues ya está todo traducido y optimizado, aunque lo interpretado es mucho más flexible para hacer pruebas puntuales. Es por ello que Python o JavaScript se usan comúnmente interpretados.

③ ¿Cuáles son las fases principales del proceso de compilación?

El código va pasando por distintos procesos que se encargan de tareas específicas:

- * **Análisis léxico:** tokens con significado, es decir, agrupa caracteres del código como los `+`, palabras reservadas, operadores, identificadores.
- * **Análisis Sintáctico:** el código debe cumplir con las reglas gramaticales del lenguaje de programación.
- * **Análisis Semántico:** Verifica que las operaciones sean válidas, es decir, que tengan sentido lógico.
- * **Optimización:** Elimina redundancias y simplifica operaciones mejorando el rendimiento del código.

• Generación de código: Convierte el código fuente en código máquina o ensamblador.

④ ¿Qué ocurre durante la fase de análisis léxico en la compilación?

RI: Es el primer paso. El compilador lee el código fuente uno a uno (carácter a carácter), en un orden específico, los agrupa en tokens relacionados.

ES: Palabra reservada "int" → se guarda ese valor int en la variable "x", entonces se convierte en un token de tipo palabra clave "x" en un identificador, "=" en un token operador y "5" en un número.

Es en este proceso donde se descartan elementos que no alteran la estructura y son inseribles, como espacios, tabulaciones, comentarios, etc. Los errores de léxico son detectados en este proceso, entonces se detiene y se marca el problema antes de continuar.

⑤ ¿Cómo se verifica la estructura gramatical del código en la fase de análisis sintáctico?

Cuando se tienen los tokens agrupados, se analiza la sintaxis, esto comprueba que el orden en que se encuentren acomodados los tokens tengan sentido y cumpla con las reglas gramaticales del lenguaje.

Es: una expresión de asignación debe tener: identificador, signo igual, expresión válida, terminar en (;). Si en algún punto esta secuencia de tokens no cumple alguna regla gramatical, arroja un error de sintaxis. Es parentesis sin cerrar, punto y coma faltante entre otros.

⑥ ¿De qué juega el análisis semántico en el proceso de compilación?

Este proceso inicia después de identificar que el código está bien escrito en el nivel sintáctico, buscando comprobar que este tenga sentido en cuanto a significado.

Es aquí donde se revisan cosas como:

- Tipos de datos compatibles, Es: intentar sumar un texto con número sin su respectiva conversión.
- Declaración de variables antes de utilizarse
- Funciones con cantidad adecuada de argumentos
- Alcance correcto de las variables.

El análisis semántico se apoya en la tabla de símbolos que funciona como una especie de registros.

⑦ ¿En qué consiste la optimización de código durante la compilación?

El objetivo es mejorarlo sin cambiar el resultado que produce el programa, es decir, el programa hace exactamente lo mismo.

pero de forma mucho más eficiente en cuanto al tiempo de ejecución, el uso de la memoria y el consumo de energía.

Algunas tareas que se hacen son: Eliminar el código muerto, es decir, código que nunca se va a ejecutar o que su resultado no se use, Propagación de constantes, es decir, calcular previamente operaciones, Optimizar bucles, asignar de manera eficiente los registros en el procesador.

⊗ ¿Qué tipos de errores puede detectar un compilador durante la fase de análisis?

Son 3 los tipos de errores que puede identificar

- Léxicos: caracter o secuencia de caracteres que el lenguaje no reconoce como válido.
- Sintácticos: la estructura del código no sigue las reglas gramaticales del lenguaje, es decir, olvidar un punto y coma, no cerrar un paréntesis, etc.
- Semánticos: Estructura correcta pero significado sin sentido, como usar variables que no se declaran, mezclar tipos de datos, funciones sin argumentos esperados.

9 ¿Qué es la generación de código en el contexto de un compilador?

Es la fase final del proceso de compilación, es aquí donde se tiene el programa completamente analizado, verificado y optimizado. Lo que faltaba es la traducción al lenguaje máquina, el que entiende el hardware.

Es en esta etapa donde se toman decisiones orientadas al hardware, como que registros del procesador usar para guardar temporalmente los valores, como organizar el acceso a la memoria, y como traducir cada instrucción intermedia en una o varias instrucciones reales de la máquina.

10 ¿Por qué es importante la optimización de código y cómo afecta el rendimiento del programa?

Básicamente un mismo programa que se escribe exactamente igual se puede traducir de muchas formas diferentes a nivel de máquina, pero no todas son igual de eficientes. Si el compilador no optimiza nada el programa puede terminar haciendo cálculos innecesarios usando más memoria de la que realmente se necesita o desperdiciando ciclos del procesador en cosas que se podían simplificar.

Cuando se optimiza bien el programa puede correr más

Design rápido, consume menos memoria y aprovecha mejor los recursos.